

Figure 5: Data dependency restricts ILP

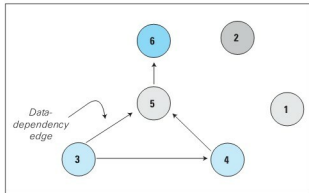


Figure 6: Data Dependency Graph (DDG)

Data dependencies between instructions are graphically represented by a Data Dependency Graph (DDG), in which:

- Each node represents an instruction (of the basic block)
 - The edges show the dependency between the instructions
- Thus, for the given six instructions in our subject code block, the DDG will be as in Figure 6.

To build a DDG:

1. Initialise the DDG with representations of the instructions.
2. Traverse the instructions of the basic block one by one, and check for data dependencies (data flow dependency/ anti-dependency/output dependency)
3. As you find dependencies, update the DDG with directed edges depicting each of them.

Step #2: Compute transitive closure

Transitive closure takes as input the DDG prepared in the previous step, and its output is a Transitive Closure Graph (TCG). We need a transitive closure to determine the indirect dependencies between the instructions. We determine the indirect dependencies from the direct dependencies.

Given that G is an n -vertex digraph (directed graph) such as the DDG shown in Figure 6, we construct the transitive closure graph of the digraph G as another n -vertex digraph (let's call it H) by adding edges to G , following this rule. In H , add an edge (i, j) directed from vertex i to j , if, and only if, there is a directed path (of any length* - 1, 2, 3, ..., $n-1$) from ' i ' to ' j ' in G .

*The number of edges in the path is referred to as the length of the path.

In Figure 6, the DDG shows that instruction 5 is dependent on Instruction 3 and Instruction 6 is dependent on Instruction

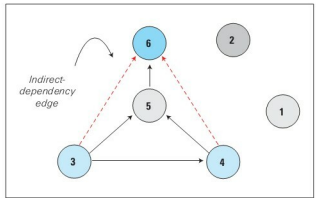


Figure 7: Transitive Closure Graph (TCG)

5. Thus, we have an indirect dependency between Instructions 3 and 6. So we add an edge between instruction Node 3 and instruction Node 6. Similarly, we have an indirect dependency between Instructions 4 and 6. We add these edges in the DDG, to obtain the TCG as shown in Figure 7 (the edges shown in red are the indirect dependency edges).

Step #3: Add architecture restrictions

Some processors may have some restrictions on which instructions can be combined in parallel. For example, we may not be able to combine two arithmetic ADD instructions in parallel. Similarly, we may not be able to combine branch instructions (like compare and jump) in parallel with any other instruction. Such restrictions are termed *architecture restrictions*.

Architecture restrictions may be represented by what I call an ILP Restriction Graph (IRG), which depicts which instructions cannot be combined in parallel. We obtain an IRG after adding architecture restrictions to the Transitive Closure Graph.

To build an IRG:

1. Initialise the IRG with the TCG (i.e., copy the nodes and edges in the TCG, as is, to the IRG).
2. Convert the graph to an undirected graph—i.e., remove any direction from the edges that are already present.
3. Add the architecture restriction edges to the IRG—these are also undirected edges, making the IRG itself an undirected graph.

The IRG is an undirected graph because the direction of edges is not relevant. When we have an architecture restriction between any two instructions A and B, whether I say we have an architecture restriction between A and B, or between B and A, is immaterial.

Let's walk through this, and convert our TCG (Figure 7) to an IRG. After converting the TCG to an undirected graph, we now add architecture restriction edges to the undirected TCG. In the above example, let's say there is the architecture restriction that we cannot combine two arithmetic operation instructions in parallel. Similarly, assume that we cannot combine two transfer instructions in parallel. That means that we have an architecture restriction between the following instructions: